



**What have we learned
in 80 years of
classical computing
that remains useful to
reason about
programming utility-
scale FTQC systems?**

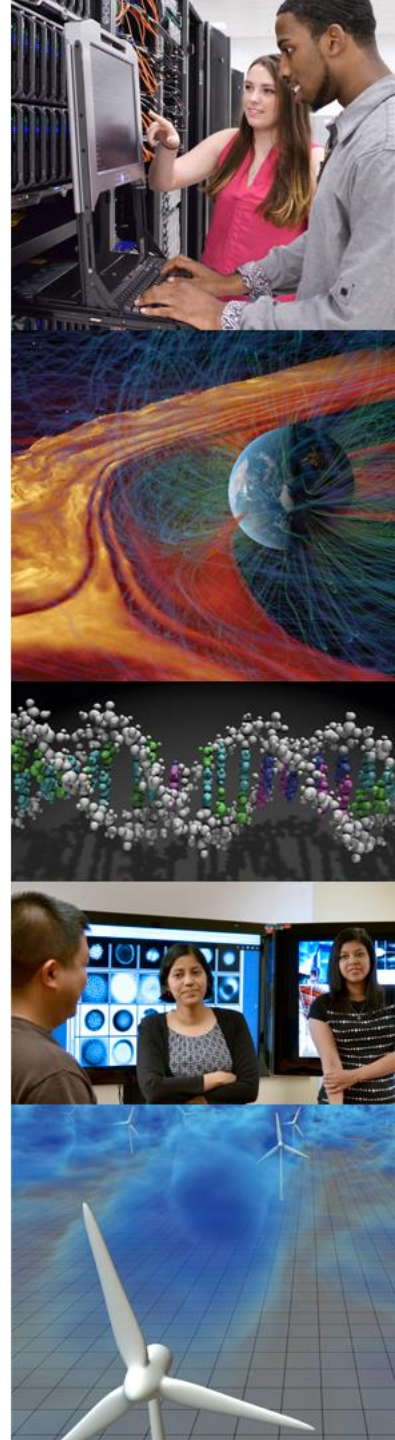


**National Center for
Supercomputing Applications**

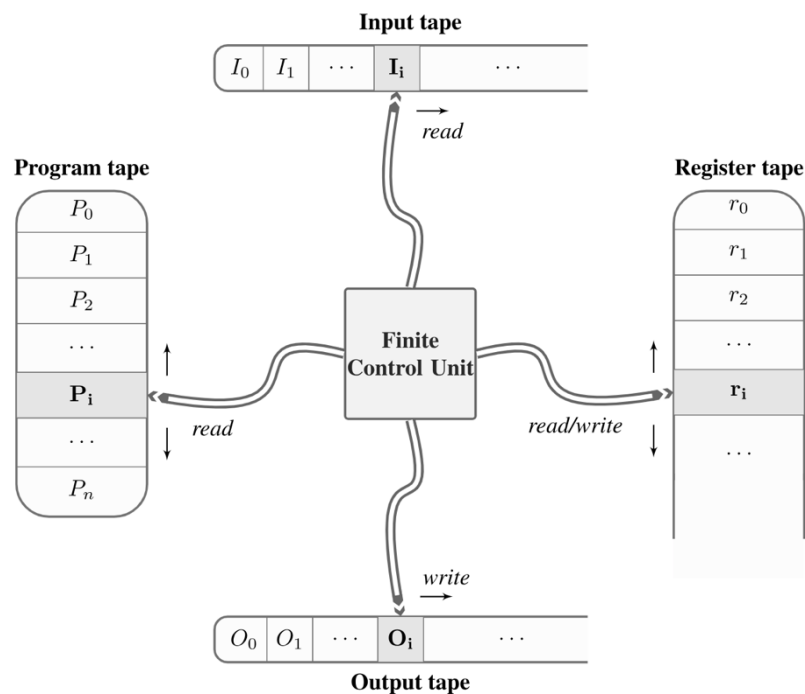
UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

Food for thought: why do build these systems and how should we help people program them?

- Marvin Minsky (1967): *“programming is a good medium for expressing poorly understood and sloppily formulated ideas”*
- Alan J. Perlis, Foreword to SICP (1985): *“a programmer should acquire good algorithms and idioms.”*
- Harold Abelson, SICP (1985): *“Programs must be written for people to read, and only incidentally for machines to execute.”*



Lesson 1: good *abstract* machines solve 80% of the algorithm development problem



Random Abstract Machines

Good abstract machines have instructions referring to functions and high-level objects. QRAM/QRASP are hardware simulators.
Núñez-Corrales, S., 2023. *arXiv:2307.08422*.

Table 2. SGRAM Machine Quantum Instruction Set

Instruction	Effect
AQBIT	$qst \leftarrow qst + 1$ $pc \leftarrow pc + 1$
CNOT tar cont inv	$QR[tar] \leftarrow tar \times cnot(cont, inv, \dots)$ $pc \leftarrow pc + 1$
GATE tar a b c d	$QR[tar] \leftarrow tar \times gate(a, b, c, d)$ $pc \leftarrow pc + 1$
HDMD tar	$QR[tar] \leftarrow tar \times gate(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$ $pc \leftarrow pc + 1$
MSRE tar	$DS[st] \leftarrow measure(tar)$ $st \leftarrow st + 1$ $pc \leftarrow pc + 1$
PHASE tar	$QR[tar] \leftarrow tar \times gate(1, 0, 0, i)$ $pc \leftarrow pc + 1$
PI tar	$QR[tar] \leftarrow tar \times gate(1, 0, 0, e^{i\pi/4})$ $pc \leftarrow pc + 1$



Lesson 2: the performance-expressiveness trade-off is *universal* and *unavoidable*

Expressive power:

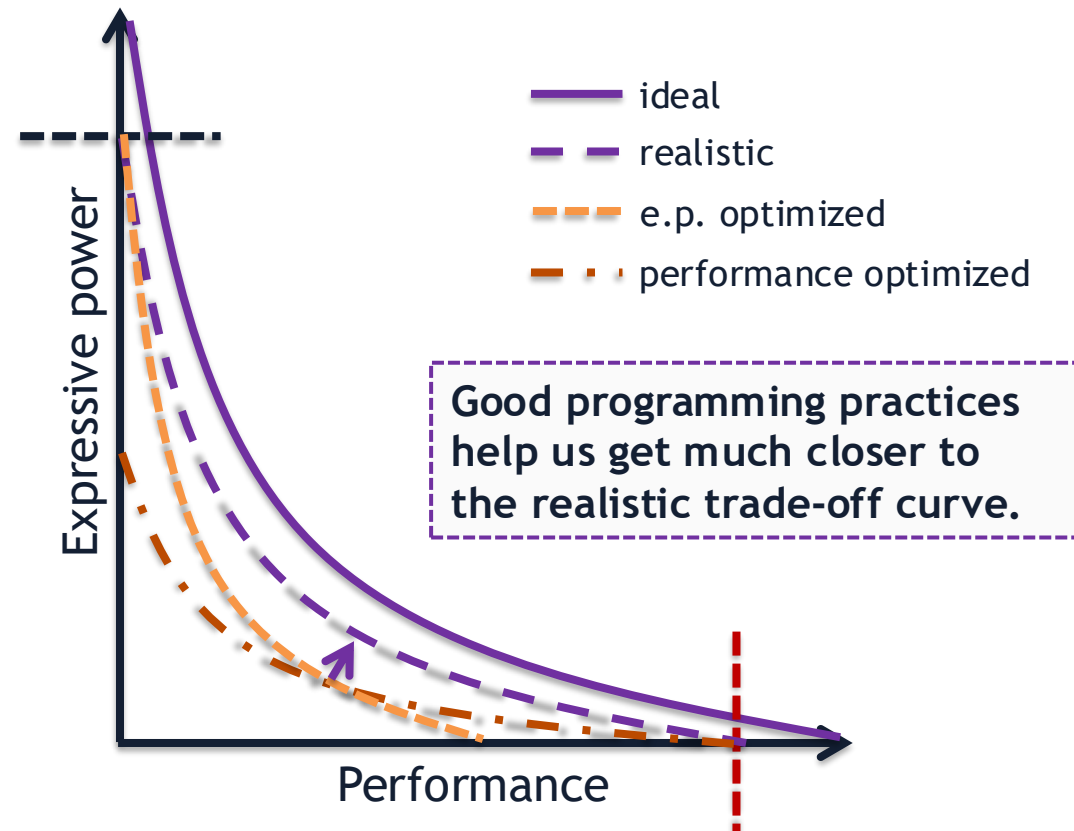
How many different problems can I describe (and solve!) with tool X?

Harder to measure:

- # of use cases
- # of instances of code reuse
- # of lines of code

Limited by:

- Theoretical bounds
- Problem features (e.g., size)
- Human needs (e.g., code maintainability and readability)



Performance:

How few resources can I use to solve a specific problem with tool X?

Easier to measure:

- Time
- Resource usage/pressure

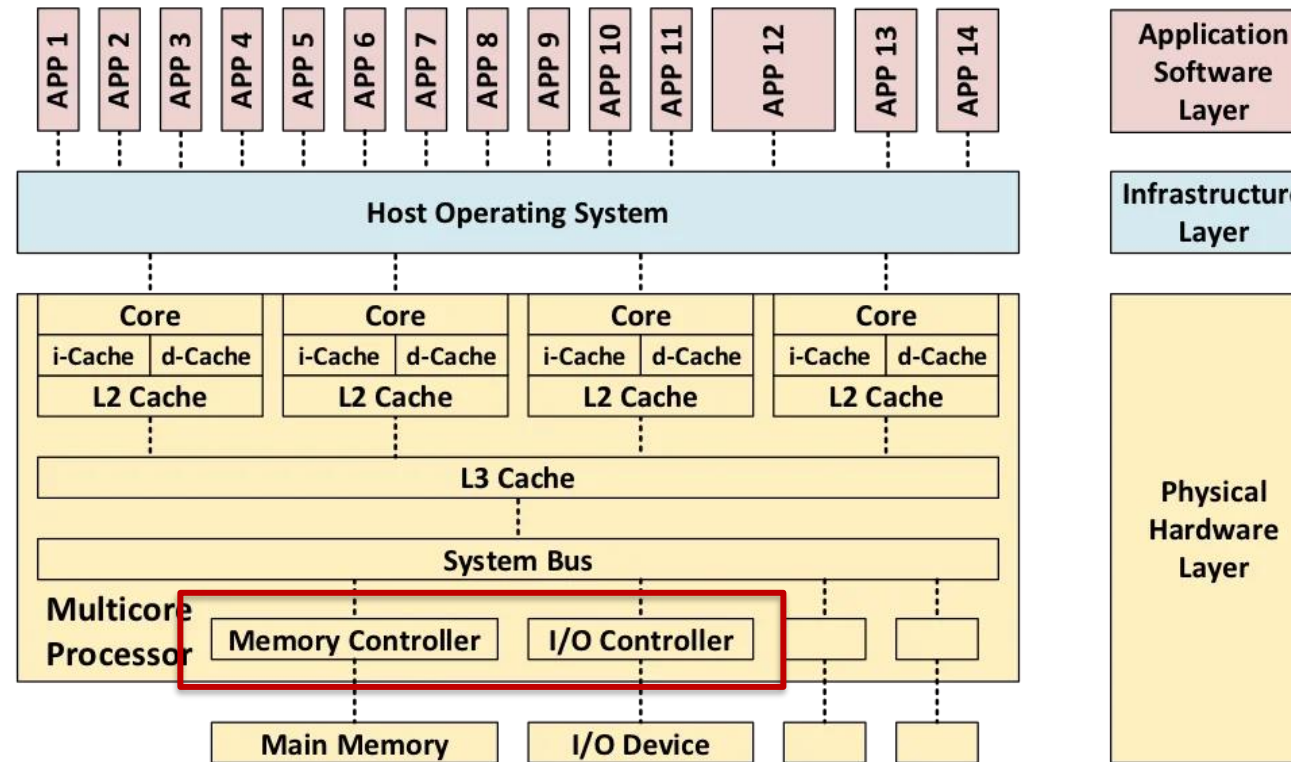
Limited by

- Technology
- Available hardware resources
- Hidden costs of compilation / interpretation + OS

We lack programming models that induce good practices in quantum computing. Pulse-level and circuit-level programming are likely not one of them.



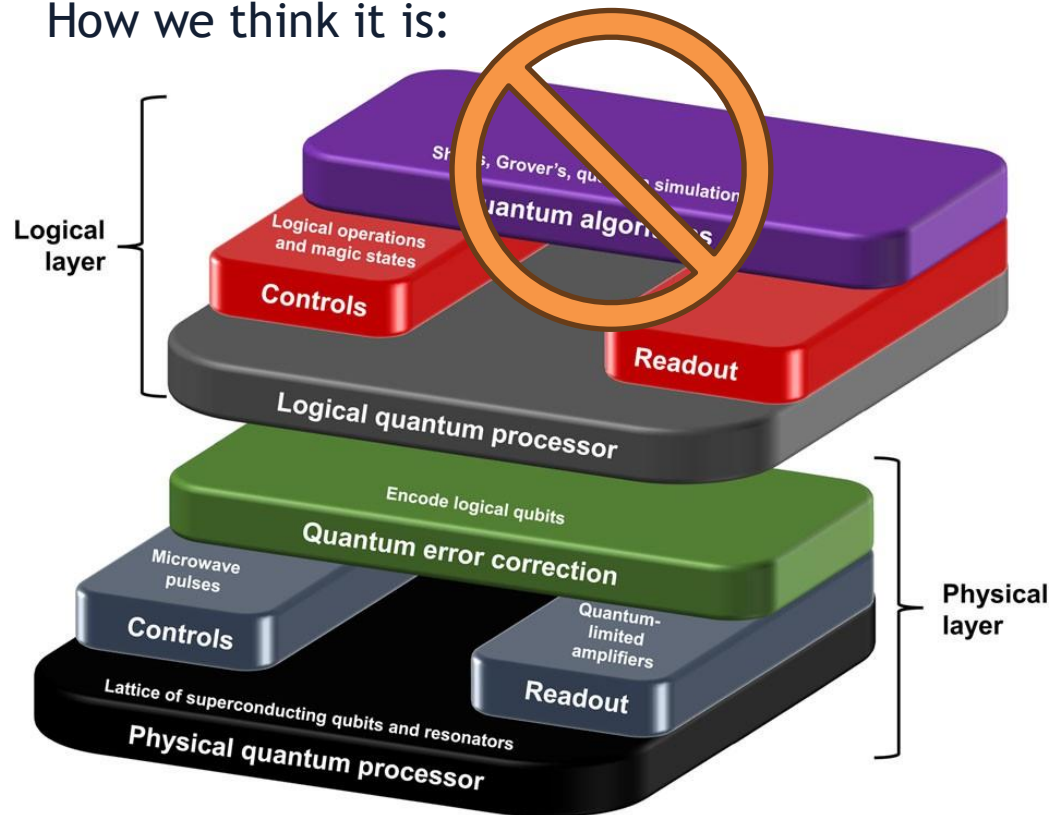
Lesson 3: “control” software becomes control hardware with time



Pulse-level synthesis and even higher quantum control primitives will likely become part of an SoC-like architecture.

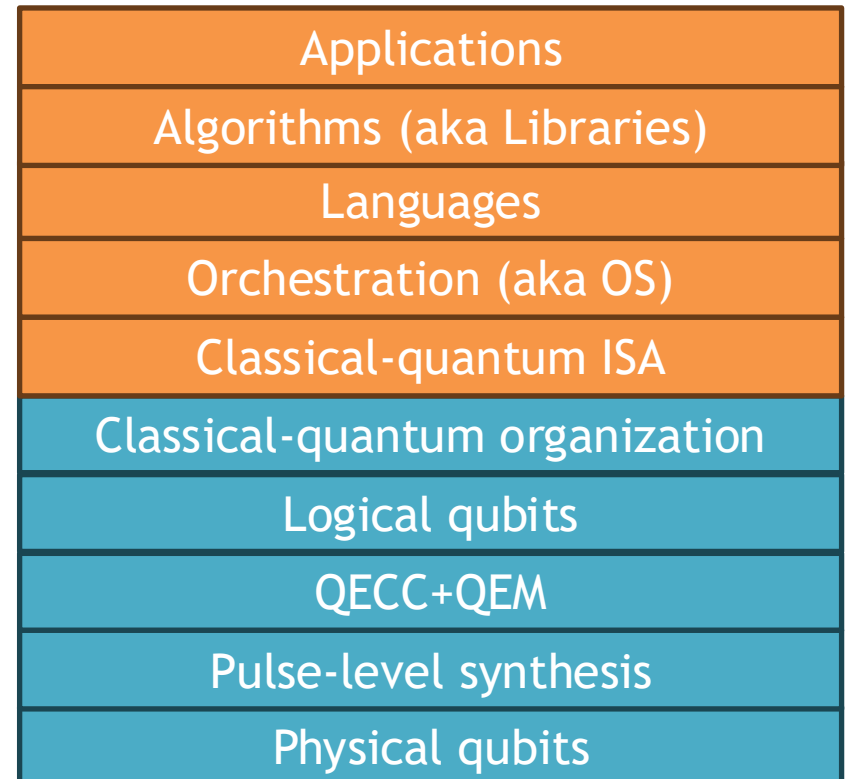
Lesson 4: good stacks enable opportunistic refinement for hw-sw co-design

How we think it is:



What we should aim for:

Hardware
detail free
zone

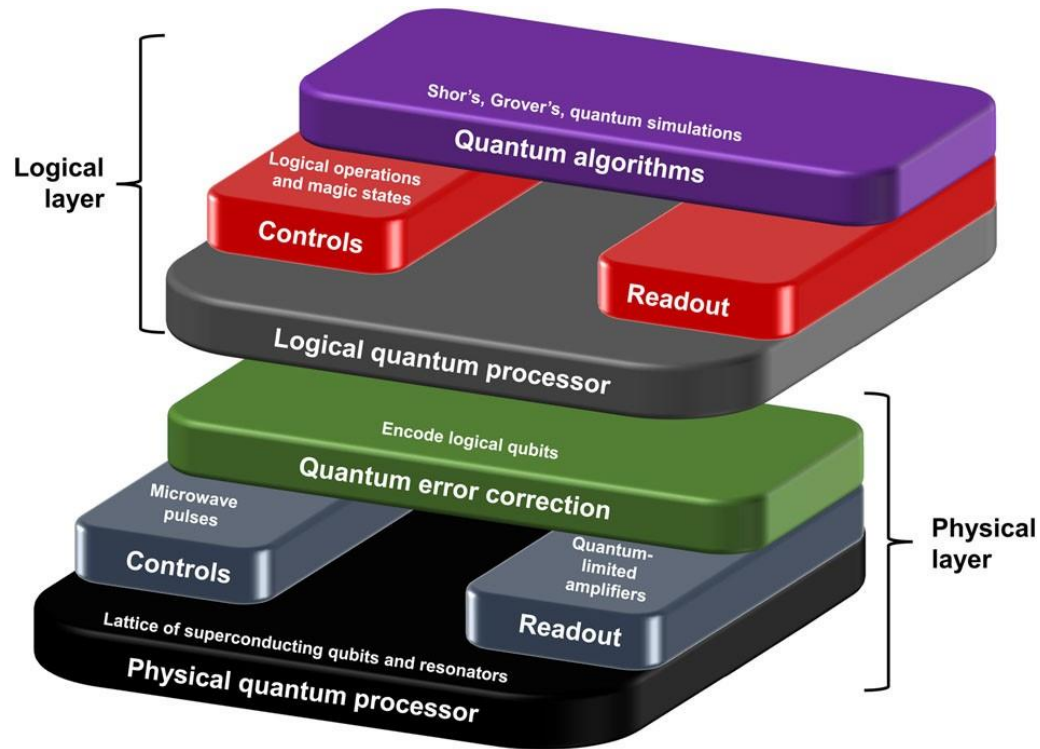


Current quantum stacks focus too much on qubit function/performance, not enough on how the interfaces across layers should communicate.



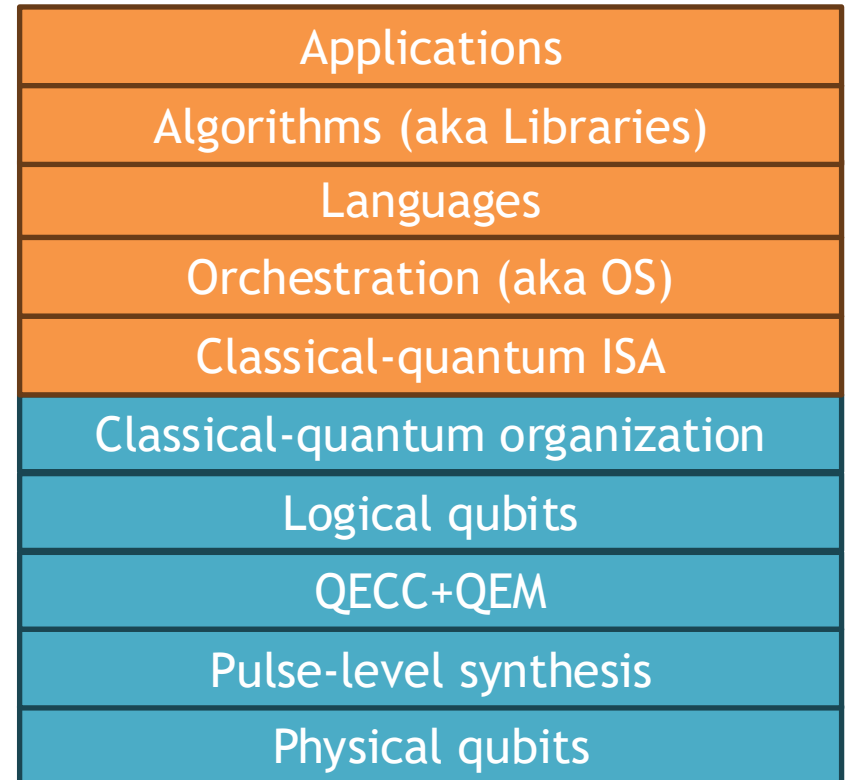
Lesson 5: good stacks separate concerns efficiently for programmers

How we think it is:



What we should aim for:

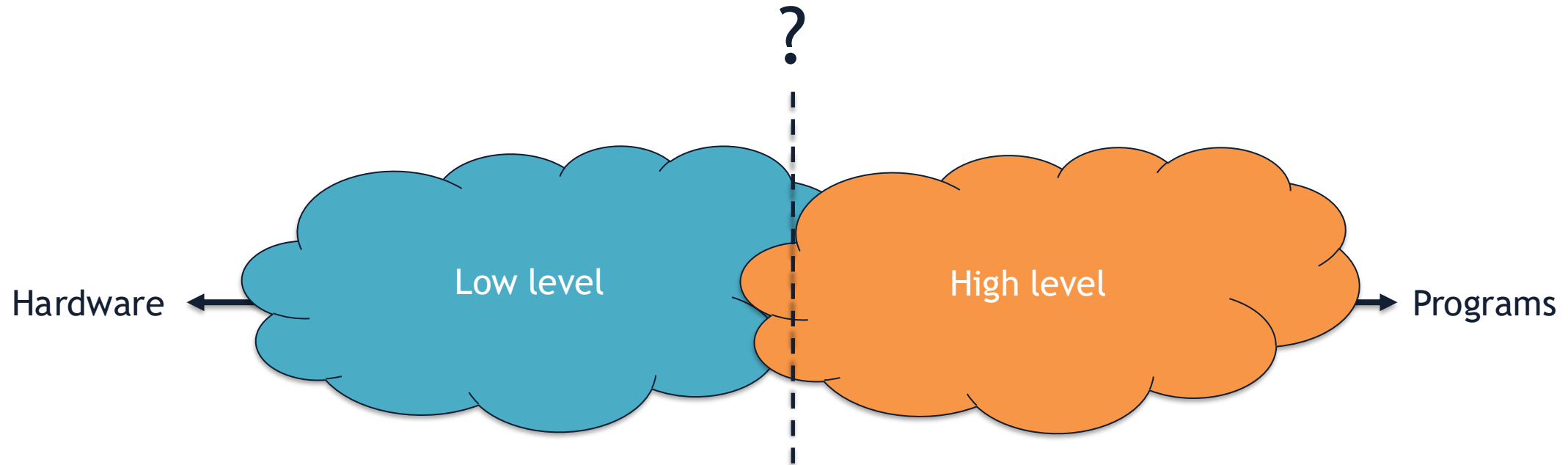
Hardware
detail free
zone



Heuristic: the difficulty of programming scales roughly proportional to the cube of the number of hardware details required to write code.



Lesson 6: circuits are not high-level constructs



- A. **Denotational semantics:** constructs isomorphic to functions within a space of objects w/ a closed algebra
- B. **Representation independent:** constructs should not vary if the “digital” representation changes
- C. **Compositionality:** the effect of large constructs is understandable from composition of smaller ones without abandoning representation independence

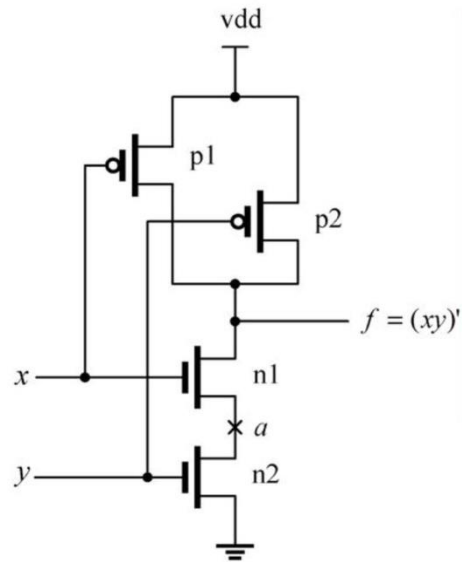
Quantum algorithms and applications will be found more quickly once we find true high-level constructs. Not there yet.

Núñez-Corrales, S., Frenkel, M. and Abreu, B., QCE 23; Di Matteo O, Núñez-Corrales S, Stęchły M, Reinhardt SP, Mattson T. arXiv:2405.13918.

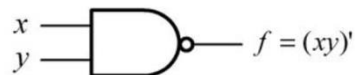


Lesson 7: generation/validation replace programming at very large hardware scales

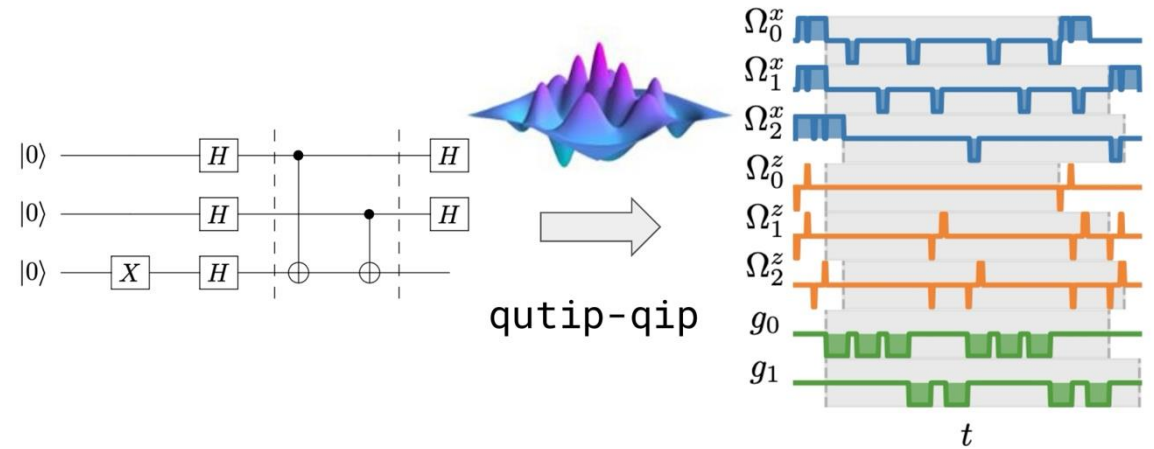
VLSI: generate and optimize -> validate



```
module my_nand (input x, y, output f);  
  supply1 vdd;  
  supply0 gnd;  
  wire a;  
  // NAND gate body  
  pmos p1 (f, vdd, x);  
  pmos p2 (f, vdd, y);  
  nmos n1 (f, a, x);  
  nmos n2 (a, gnd, y);  
endmodule
```

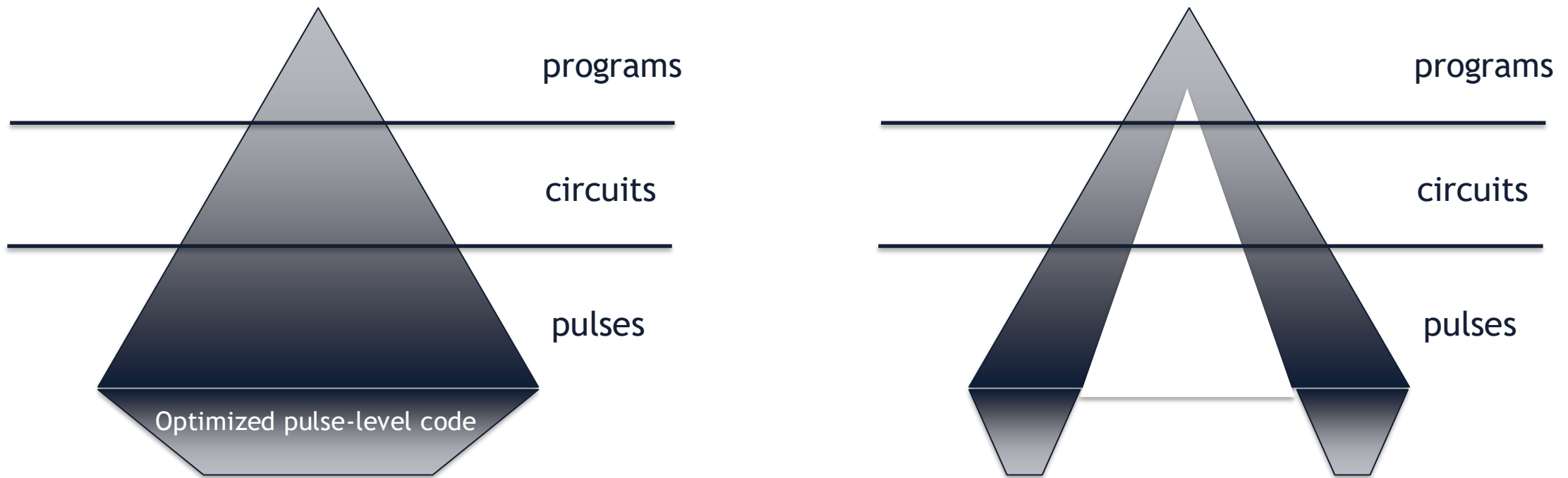


VLQI: very large quantum integration



Utility-scale, fault-tolerant quantum computers pose a wicked control problem for humans. Most likely, many of these are NP-HARD. VLQI will be self-bootstrapping.

Lesson 8: resist to optimize within differences that make no difference



As quantum computers become larger ($>10^4$ logical qubits), optimizations must occur as high up as possible.



Lesson 9: modularity and indirection organize complexity

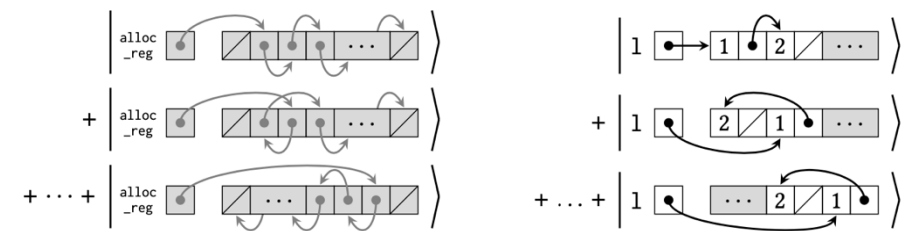
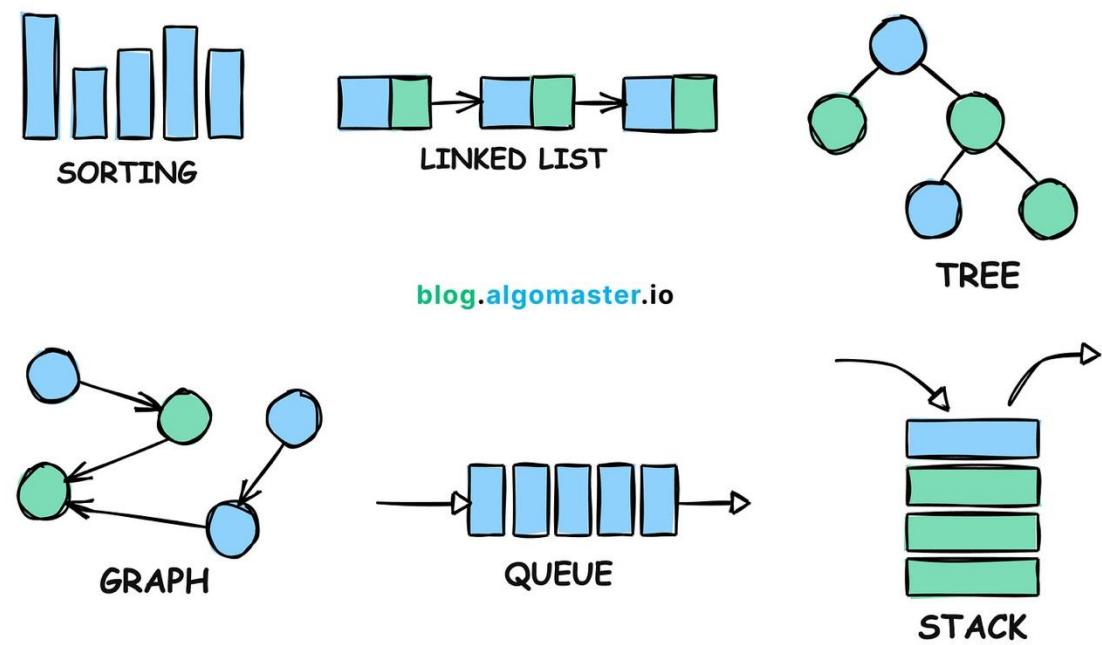


Fig. 17. Result of symmetrization on the initial program state from Figure 12 (normalizing amplitudes not shown). The symmetrized free list exists in a superposition of all possible permutations.

Fig. 18. Unique physical representation of state $1 \leftrightarrow [1, 2]$ (normalizing amplitudes not shown), which stores data in a superposition of all possible allocation sites and is history-independent.

Data Structure	Reversible	Recursion	Mutation	Complexity	LoC	Qubits	Gates
List							
- length	Yes	Yes	No	$O(n)$	20	$34n + 32$	$23n + 3$
- sum	Yes	Yes	No	$O(n)$	20	$34n + 40$	$21n + 3$
- find_pos	Yes	Yes	No	$O(n)$	20	$42n + 31$	$19n + 3$
- remove	Yes	Yes	Yes	$O(n)$	48	$26n + 56$	$42n + 3$
Stack (list)							
- push_front	Yes	No	Yes	$O(1)$	8	40	4
- pop_front	Yes	No	Yes	$O(1)$	8	48	4
Queue (list)							
- push_back	Yes	Yes	Yes	$O(n)$	21	$34n + 32$	$24n$
- pop_front	Yes	No	Yes	$O(1)$	8	48	4
String (word)							
- is_empty	Yes	No	No	$O(1)$	2	25	3
- length	Yes	No	No	$O(1)$	2	24	1
- get_prefix	Yes	No	No	$O(k)$	8	$11k$	52
- get_substring	Yes	No	No	$O(k)$	8	$12k$	54
- get	Yes	No	No	$O(k)$	7	$6k + 1$	19
- is_prefix	Yes	Yes	No	$O(\text{poly}(k))$	26	$k^2 + 11k$	$98k + 3$
- num_matching	Yes	Yes	No	$O(\text{poly}(k))$	42	$k^2 + 13k + 4$	$110k + 127$
- equal	Yes	No	No	$O(k)$	8	$6k + 3$	5
- concat	Yes	No	No	$O(k)$	9	$11k$	8
- compare	Yes	Yes	No	$O(\text{poly}(k))$	27	$5k^2 + 12k$	$108k + 3$
Set (radix tree)							
- insert	Yes	Yes	Yes	$O(\text{poly}(k))$	136	$13k^2 + 21k + 9$	$1440k^2 + 5056k$
- contains	Yes	Yes	No	$O(\text{poly}(k))$	334	$17k^2 + 18k + 2$	$784k^2 + 1612k + 1$
Set (hash table)*							
- insert	Yes	Yes	Yes	$O(n)$	63	$52n + 72$	$68n + 15$
- contains	Yes	Yes	No	$O(n)$	7	$52n + 81$	$136n + 39$

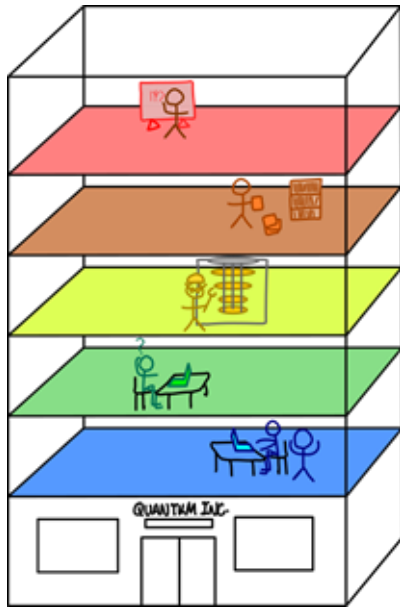
* Hash table-based sets are not history-independent.

Few quantum data structures, more needed to scale up to utility-scale systems.

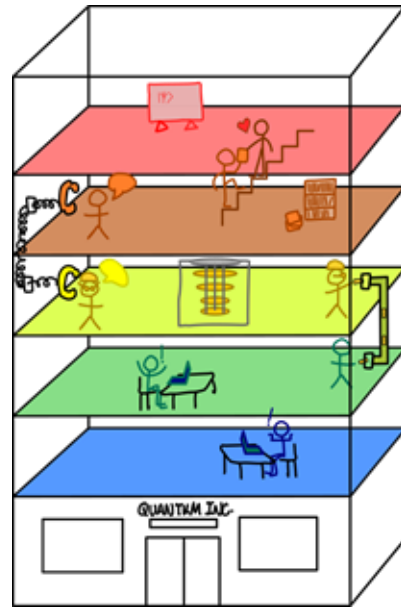
Yuan, C. and Carbin, M., 2022. OOPSLA2.



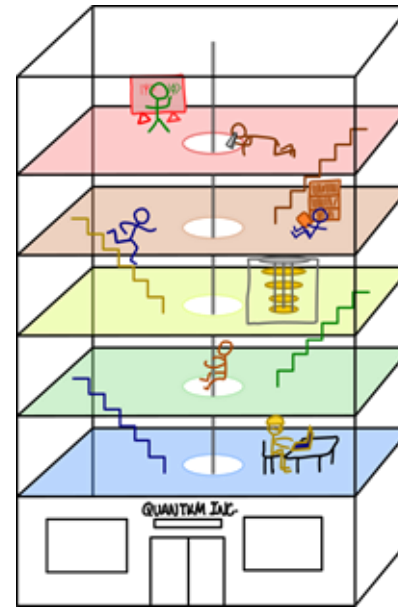
Why do we want good abstractions?



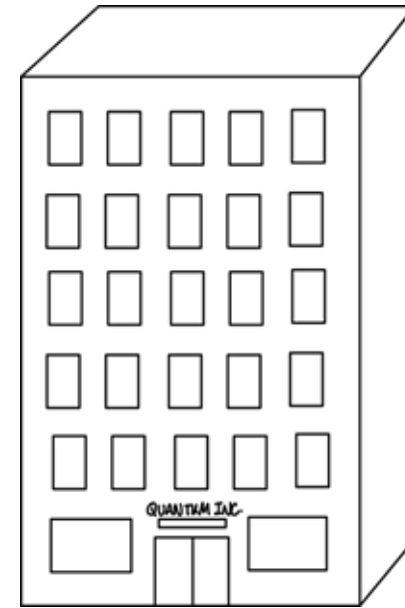
Separation of
concerns



Well-defined
interactions
between adjacent
layers



Opportunistic
refinement

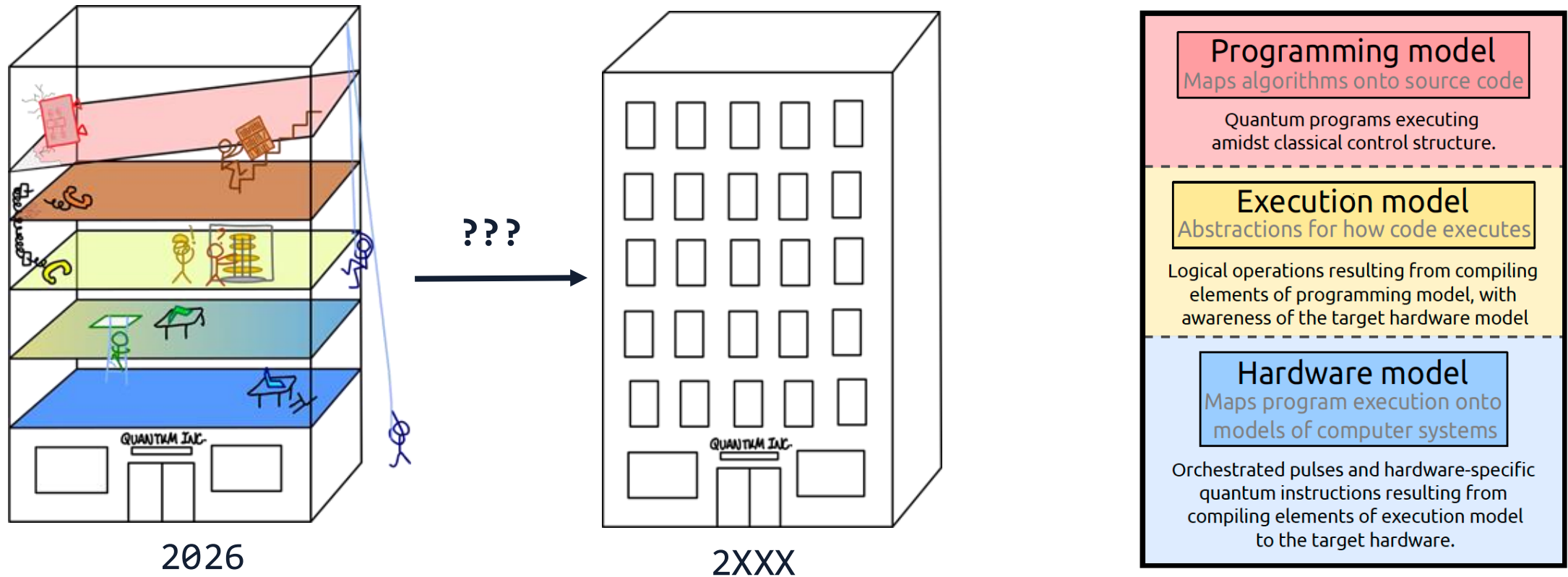


...

Di Matteo, O., Núñez-Corrales, S., Stęchły, M., Reinhardt, S.P. and Mattson, T., 2024, September. An abstraction hierarchy toward productive quantum programming. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (Vol. 1, pp. 979-989). IEEE.



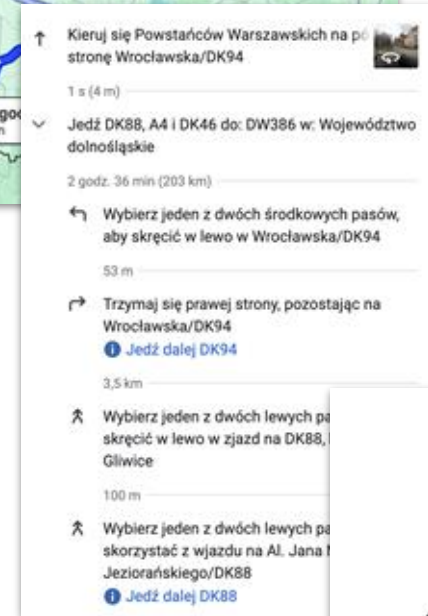
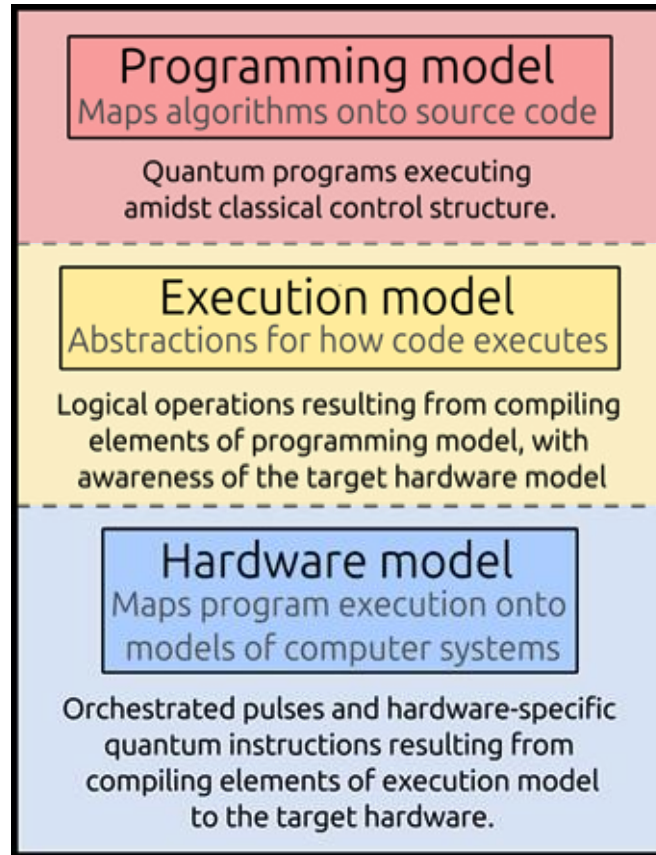
The state of quantum programming today



Di Matteo, O., Núñez-Corrales, S., Stęchły, M., Reinhardt, S.P. and Mattson, T., 2024, September. An abstraction hierarchy toward productive quantum programming. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (Vol. 1, pp. 979-989). IEEE.

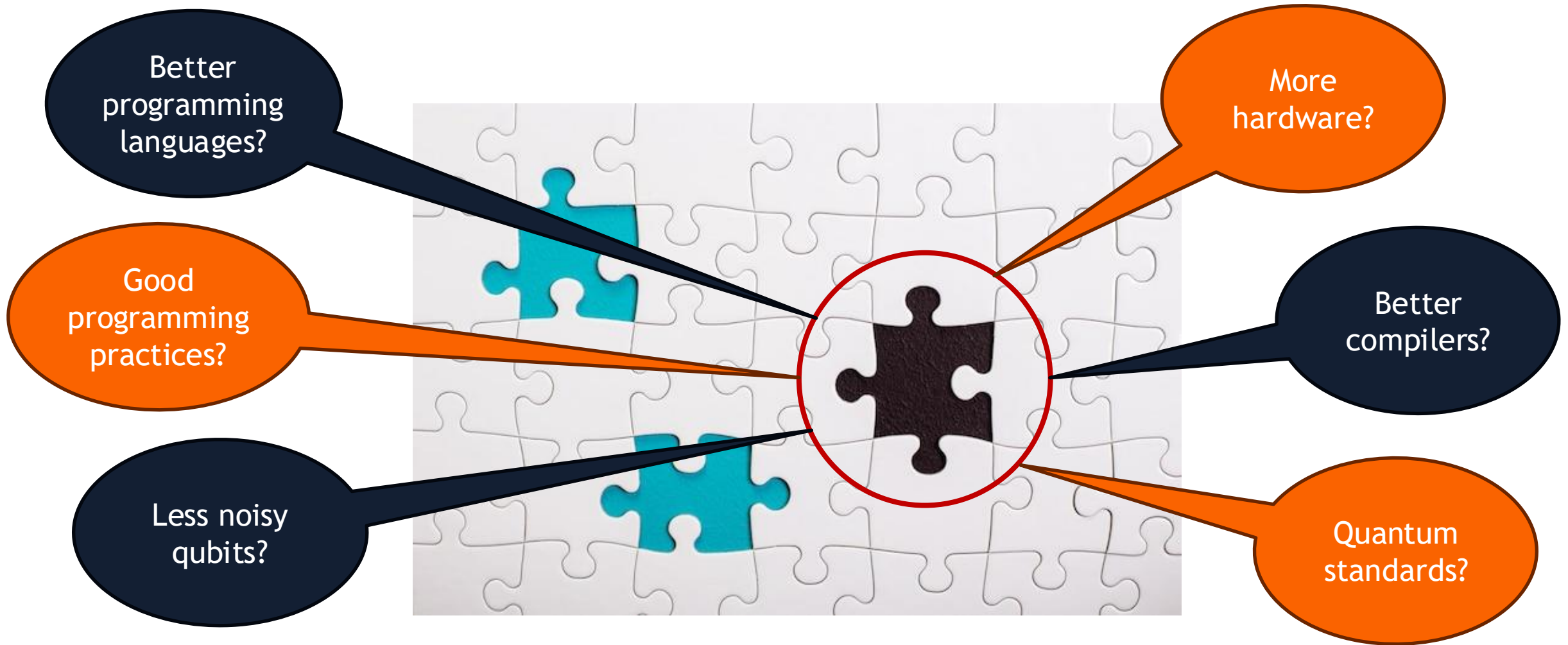


Abstractions mediate effectiveness

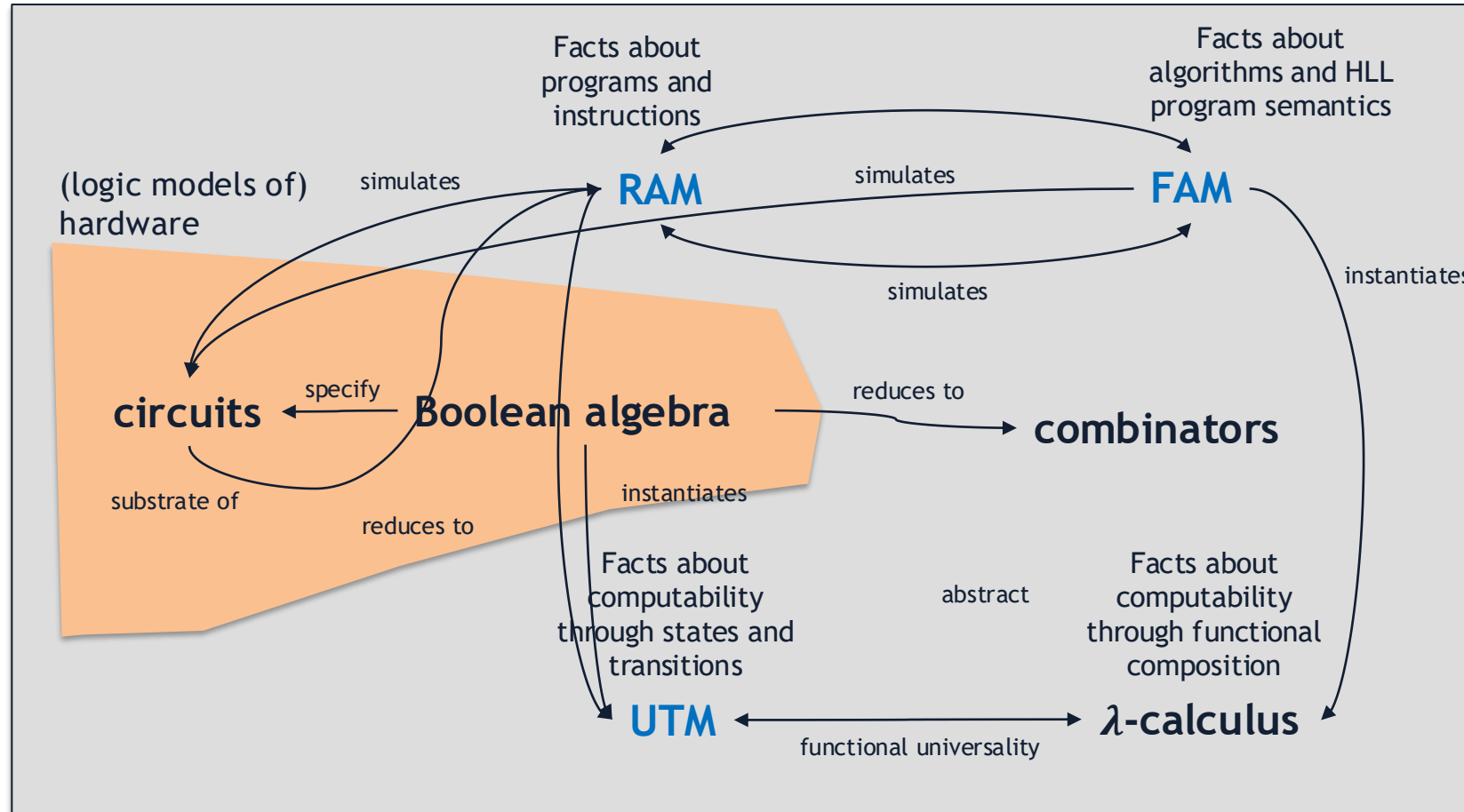


Di Matteo, O., Núñez-Corrales, S., Stęchły, M., Reinhardt, S.P. and Mattson, T., 2024, September. An abstraction hierarchy toward productive quantum programming. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (Vol. 1, pp. 979-989). IEEE.

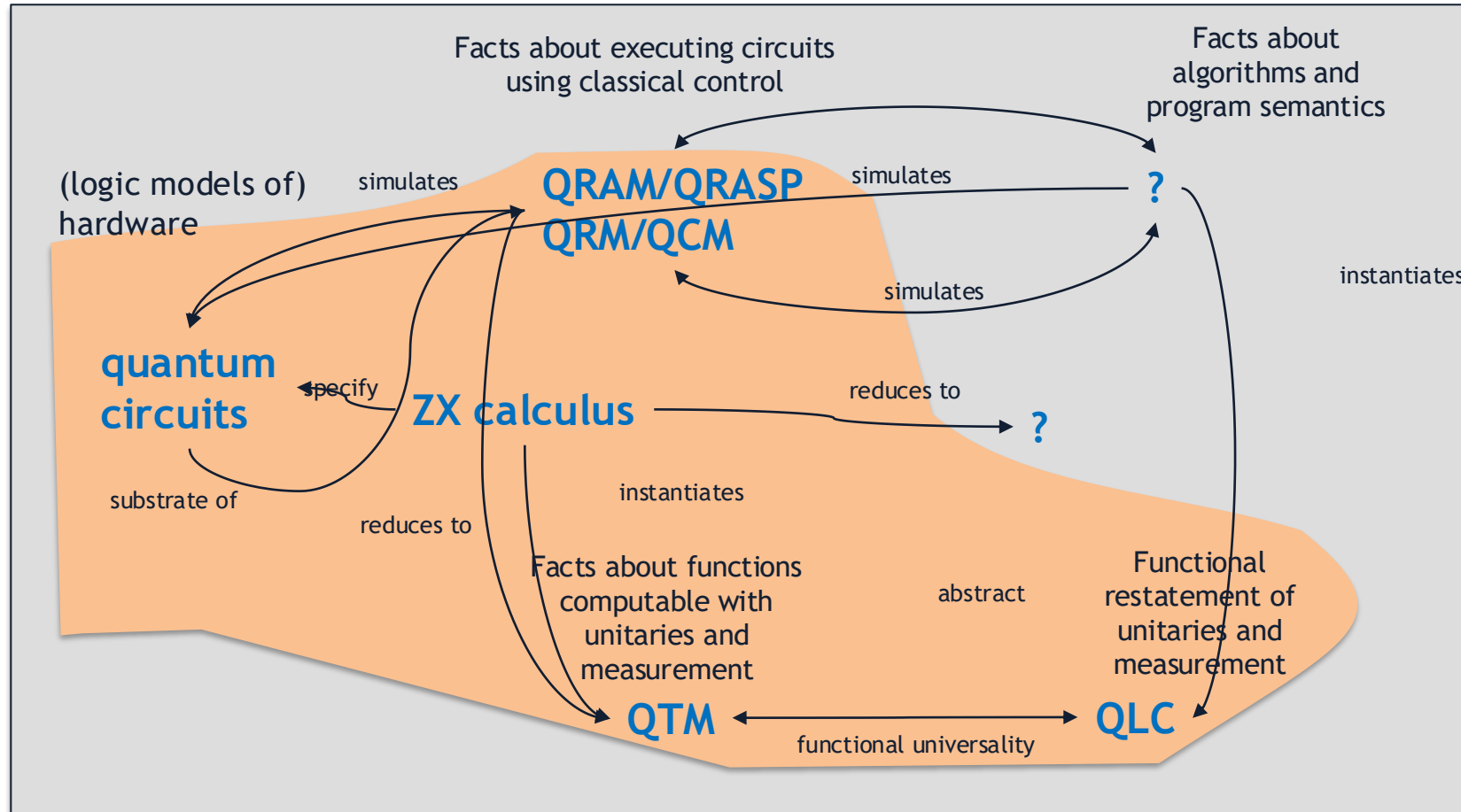
But: something really important is missing...



Abstract machines serve different purposes



All quantum abstract machines so far are only great circuit execution machines



Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

Santiago Núñez-Corrales (Illinois)

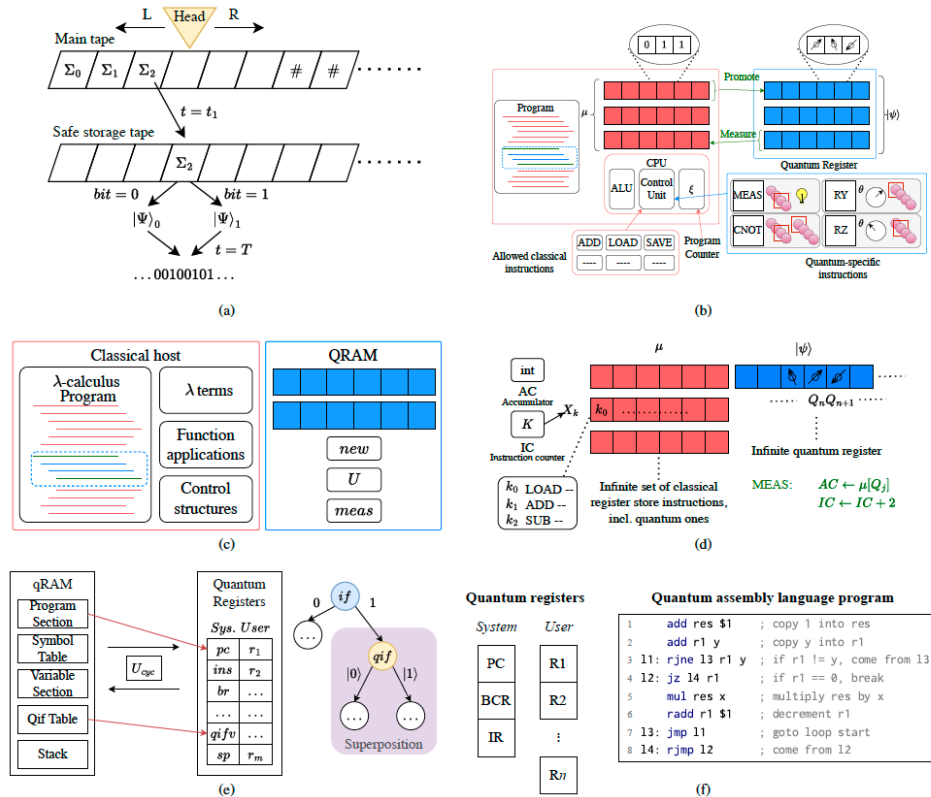


Fig. 1. Graphical representations of: (a) the Quantum Turing Machine [15], (b) the Quantum Random Access Machine [16], [17], (c) the Quantum Lambda Calculus Machine [18], (d) the Quantum Random Access Stored Program Machine [19], (e) the Quantum Register Machine [20], and (f) the Quantum Control Machine (with the code example taken verbatim from the QCM paper) [21]

TABLE I
ANALYSIS OF PREVAILING QUANTUM ABSTRACT MACHINES

Criterion	Description	QTM [19]	QRAM [19]	QRASP [19]	QRM [20]	QCM [21]	QLC [37]
1	Turing-complete & universal	✓	✓	✓	✓	✓	✓
2	Finite symbolic state				✓	✓	
3	Symbolic denotational semantics						
4	Representation-independent data types						
5	Stable instruction set architecture						
6	Verifiable formal content	✓	✓	✓	✓	✓	✓
7	Classical-quantum regularity	✓			✓†	✓†	
8	Compact instruction representation		✓	✓	✓	✓	✓
9	Degeneracy of implementation	✓	✓	✓	✓	✓	✓
10	Predictable procedural composability		✓	✓	✓	✓	
11	Intrinsic ensemble semantics	✓	✓	✓	✓	✓	
12	Resource-constructible functions	✓	✓	✓	✓	✓	✓
13	Standard instruction cycle		✓	✓	✓	✓	
14	Classical control flow			✓	✓	✓	✓
15	Quantum/hybrid control flow				✓	✓	
Total		6✓	8✓	9✓	11✓	11✓	6✓

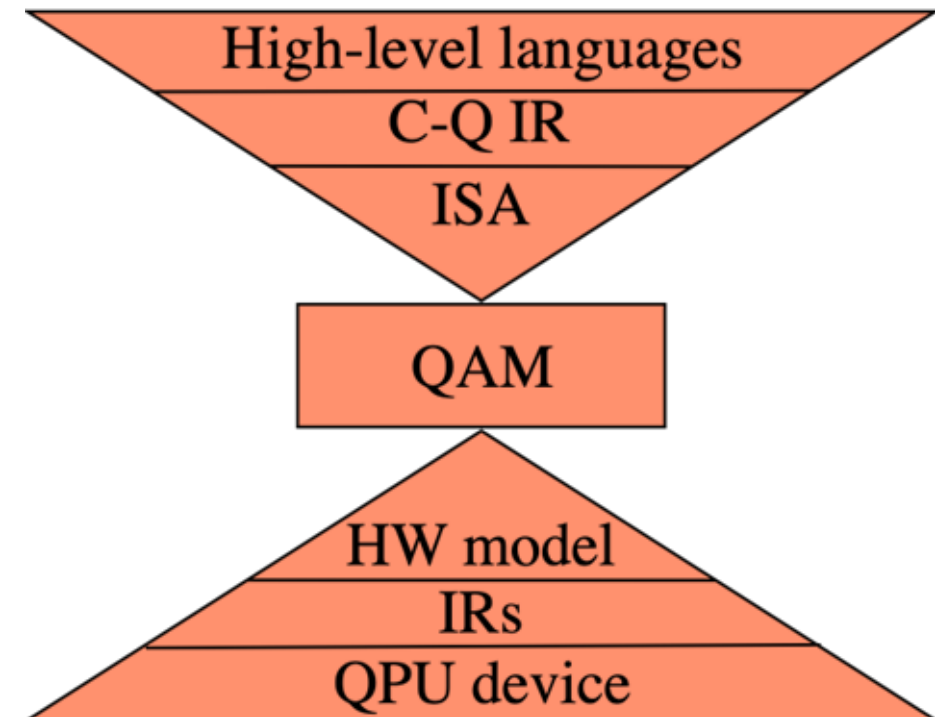
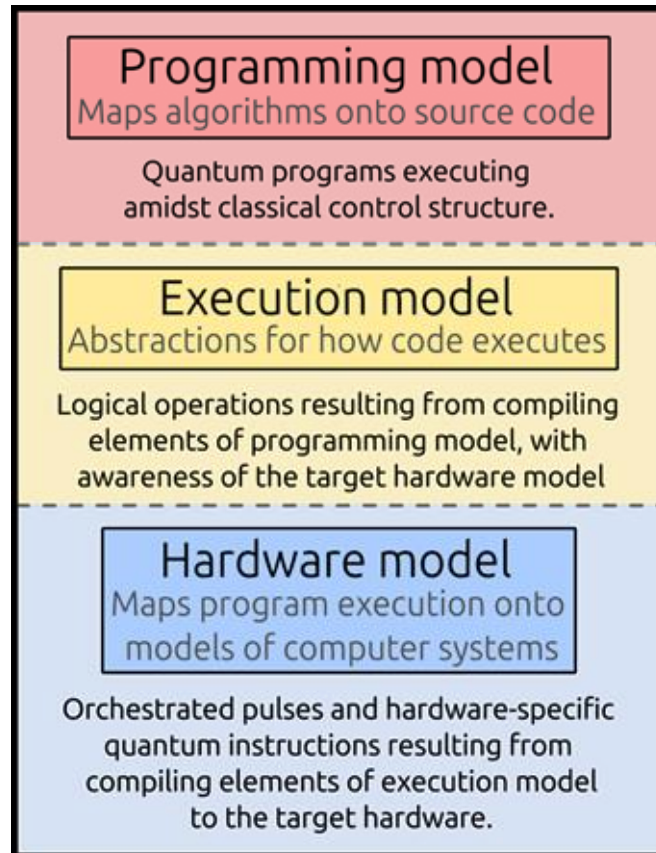
† partial satisfaction due to explicit mention of unitary gates

Núñez-Corrales, S., Di Matteo, O., Dumbell, J., Edwards, M., Giusto, E., Pakin, S., Stirbu, V.Stęchły, M. (2025, submitted). Productive Quantum Programming Needs Better Abstract Machines. *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE/arXiv (submitted). -> **IIDAI funded!**

Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

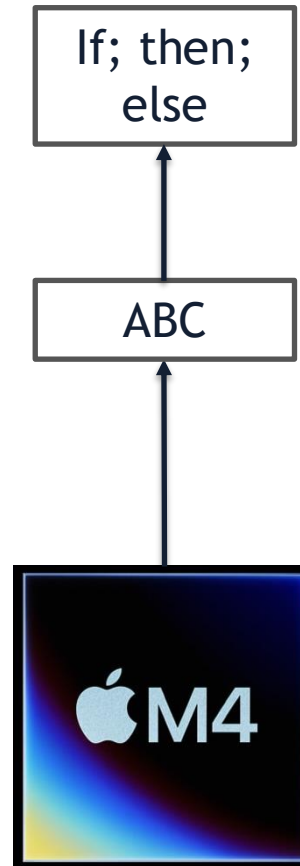
Santiago Núñez-Corrales (Illinois)



Di Matteo, O., Núñez-Corrales, S., Stęchły, M., Reinhardt, S.P. and Mattson, T., 2024, September. An abstraction hierarchy toward productive quantum programming. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (Vol. 1, pp. 979-989). IEEE.

Vision – The Challenge

- Need to break the intuition that programming models should scale with hardware resources
- We rather need **scale-free programming models** w.r.t. qubit/gate counts
- Separation of **manufacturability** from **programmability**
- We've been here before (i.e. VLSI), we will need to get there again (i.e., FTQC)
- This requires separating the denotation of quantum programs from compilation into executable physical circuits, from their execution during runtime
- **Quantum programs that fit in the mind of application writers accelerate utility**
- **Number of successful applications inversely proportional to number of QISE PhDs needed to develop them**



$O(1)$ control flow patterns

10^3 instructions
 $\sim 10^2$ used 90% of time

$\sim 2.8 \times 10^{10}$ transistors

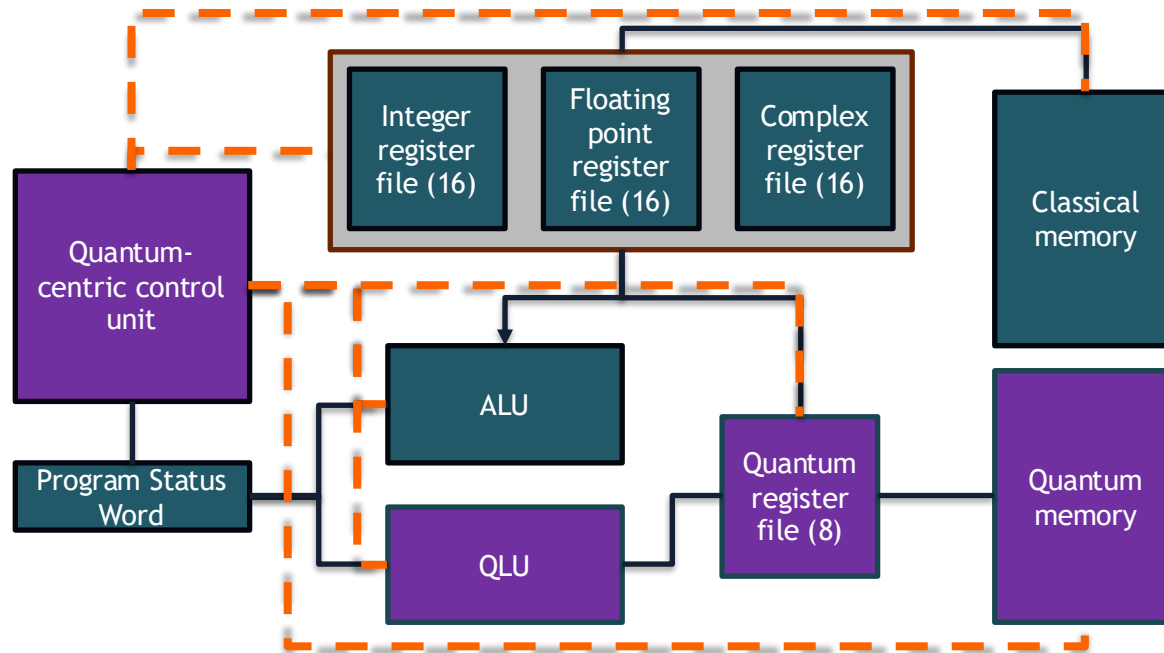
7-8 orders of magnitude

Abstract machines organize complexity into event schedules with varied hardware

Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

Santiago Núñez-Corrales (Illinois)



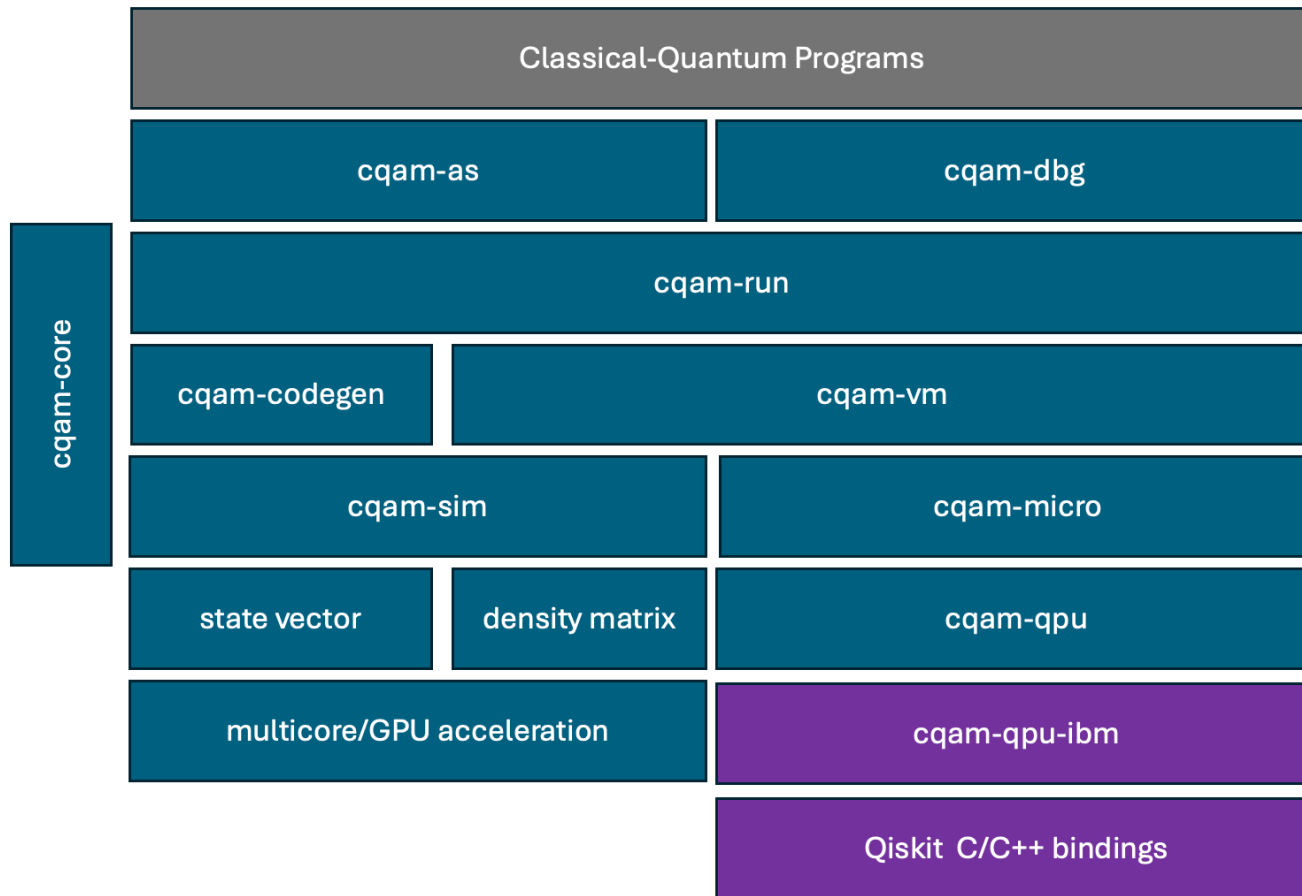
reserved	trap	hybrid	quantum	classical			
Flag	Meaning	Flag	Meaning	Flag	Meaning		
HT	Halt	H	Hybrid block	QF	Conditional	ZF	Zero
TI	Interrupt	KF	Forked path	LF	Low fidelity	NF	Negative
		RF	Merged path	MF	Measure	CF	Carry
		TF	C/Q type cast	FF	Fault	OF	Overflow
		VF	Converged	EF	Entangled	XF	Undefined
				IF	Interfered		
				SF	Superposed		
				UF	Undefined		

Núñez-Corrales, S. (in preparation). A Classical-Quantum Abstract Machine. ACM Transactions on Quantum Computing/ arXiv.

Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

Santiago Núñez-Corrales (Illinois)



Product Features:

- Consistent Instruction Set Architecture
- Written in idiomatic Rust with crates
- Multicore Rayon parallelization
- Execution from source or binary generation
- Full assembler and disassembler
- Classical-quantum CLI debugger
- Detailed noise models for density matrix simulator
- Automatic Bayesian shot estimator
- Configurable backends with API keys
- Calibration-aware qubit selection
- Full use of Qiskit C/C++ bindings
- Visual Studio Syntax Highlight package

Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

Santiago Núñez-Corrales (Illinois)

```
# --- Prepare initial state: uniform superposition ---
QPREP Q0, 0      # Q0 = uniform superposition

# --- QPE rotation loop: apply U^(2^k) for k = 0, 1, ..., R1-1 ---
# Each iteration doubles the rotation angle, simulating controlled-U^(2^k)
FLDI F2, 1      # F2 = theta_0 = base angle

LABEL: qpe_loop

# Apply rotation with current angle
QKERNELF DROT, Q0, Q0, F2, F3 # Q0 = Rotate(Q0, angle=F2)

# Double the angle for next iteration: F2 = F2 * 2
FMUL F2, F2, F1

# Increment counter
IADD R0, R0, R2
ISUB R1, R1, R2

# Continue if R1 > 0
JIF R1, qpe_loop

# --- Apply QFT to extract phase information ---
QKERNEL QFFT, Q1, Q0, R3, R3 # Q1 = QFT(rotated state)

# --- Destructive measurement for full distribution ---
QOBSERVE H1, Q1      # H1 = full distribution (destructive)

# Print measurement histogram (top-10 for 16-qubit state)
ISTR R2, 999      # Save R2 (qubit count) to CMEM[999]
ILDI R0, 1      # H1 = phase estimation distribution
ILDI R1, 3      # Mode 3: top-K
ILDI R2, 10     # Show top 10
ECALL PRINT_HIST
ILDMM R2, 999    # Restore R2 from CMEM[999]
```



```
cargo run --bin cqam-as -- --assemble examples/intermediate/phase_estimation.cqam
-o qpe.cqb
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.17s
Running `target/debug/cqam-as --assemble examples/intermediate/phase_estimat
ion.cqam -o qpe.cqb`
Assembled 64 instructions (3 labels, entry at word 0)
Wrote 268 bytes to qpe.cqb

~/Research/cqam git:(main) 1 file changed, 10 insertions(+) (0.051s)
hexdump qpe.cqb
00000000 5143 4d41 0001 0000 0040 0000 03e8 0c00
00000010 001a 0c10 0001 0cf0 f000 0211 0000 2f20
00000020 0402 0c00 0023 0c10 0001 0cf0 f000 0211
00000030 0000 5820 0000 2f20 0001 1600 0002 1610
00000040 0000 1620 0000 1630 0000 0c00 0004 0c10
00000050 0001 0c20 0000 0c30 0000 3000 0001 1620
00000060 0000 2c00 a460 4100 1000 1422 2000 0100
00000070 2000 0211 0015 2810 4660 3120 0000 3224
00000080 03e7 0e20 0001 0c00 0003 0c10 000a 0c20
00000090 0000 2f50 03e7 0d20 5000 3b14 6800 3b15
000000a0 6000 3b15 0000 2365 0006 1670 7000 1486
000000b0 0000 5860 0001 0c70 0000 0c80 0100 2c00
000000c0 6000 0f98 0032 2890 7000 0177 2000 0188
000000d0 002c 2700 0200 2c00 0000 2397 9000 1588
000000e0 0000 1840 0001 1850 0002 1880 4000 0125
000000f0 0002 1710 0425 0c00 002b 0c10 0001 0cf0
00001000 f000 0211 0000 2f20 0000 2b00
00001010 c
```



```
cargo run --bin cqam-run --features ibm -- examples/intermediate/phase_estimation
.cqam --backend ibm --qpu-device ibm_kingston --qpu-shots 1024
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.18s
Running `target/debug/cqam-run examples/intermediate/phase_estimation.cqam -
-backend ibm --qpu-device ibm_kingston --qpu-shots 1024`
=== Phase Estimation ===
Inputs: qubits=16, base_angle=1.0
```

Qiskit High: achieving expressiveness in quantum programming

Thrust: Quantum

Santiago Núñez-Corrales (Illinois)

Workloads

Filtered by region: us-east ⓘ

Track the status and results of the workloads you have run on IBM quantum processing units.

Need help?
[View docs →](#)

🔍 Search by ID or tag

My workloads ✕ [Reset filters](#)

☐	ID	Status	Instance	Mode
☐	d7f9bk56agrc7...	⌚ Pending	IIDAI Quantum Thrust	Job
☐	d7f976i1u7fs73...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f947td4inc73...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f945a1u7fs7...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f942td4inc73...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f94056agrc7...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f93ti1u7fs73...	✅ Completed	IIDAI Quantum Thrust	Job
☐	d7f93r56agrc73...	✅ Completed	IIDAI Quantum Thrust	Job

IBM Quantum Platform

Diagram

Qasm

Qiskit

q[39]

RZ

(1, 570796326794897)

√X

RZ

(1881, 570796326794897)

q[50]

RZ

(1, 570796326794897)

√X

RZ

(1921, 570796326794897)

q[51]

RX

(0, 376755297750938)

RZ

(-1, 570796326794897)

RZ

(39721, 570796326794897)

q[52]

RZ

(1, 570796326794897)

√X

RZ

(39721, 570796326794897)

q[53]

RZ

(1, 570796326794897)

√X

RZ

(43441, 570796326794897)

q[54]

RZ

(5, 570796326794897)

√X

RZ

(445961, 570796326794897)

q[55]

RX

(-2, 9587723626792827)

RZ

(-1, 570796326794897)

RX

(-1, 570796326794897)

RX

(0, 785398263397440)

RX

(-0, 785398263397440)

RX

(-1, 378897245096172)

RZ

(-1, 570796326794897)

√X

√X

RZ

(1, 570796326794897)

√X

RZ

(1, 570796326794897)

q[58]

RZ

(1, 570796326794897)

√X

RZ

(15361, 570796326794897)

q[59]

RZ

(1, 570796326794897)

√X

RZ

(122851, 570796326794897)

q[71]

RZ

(1, 570796326794897)

√X

RZ

(7961, 570796326794897)

q[72]

RZ

(1, 570796326794897)

√X

RZ

(491, 570796326794897)

q[73]

RZ

(1, 570796326794897)

√X

RZ

(241, 570796326794897)

q[74]

RZ

(5, 570796326794897)

√X

RZ

(61, 570796326794897)

q[75]

RZ

(1, 570796326794897)

√X

RZ

(16, 570796326794897)

q[79]

RZ

(1, 570796326794897)

√X

RZ

(121, 570796326794897)

q[93]

RX

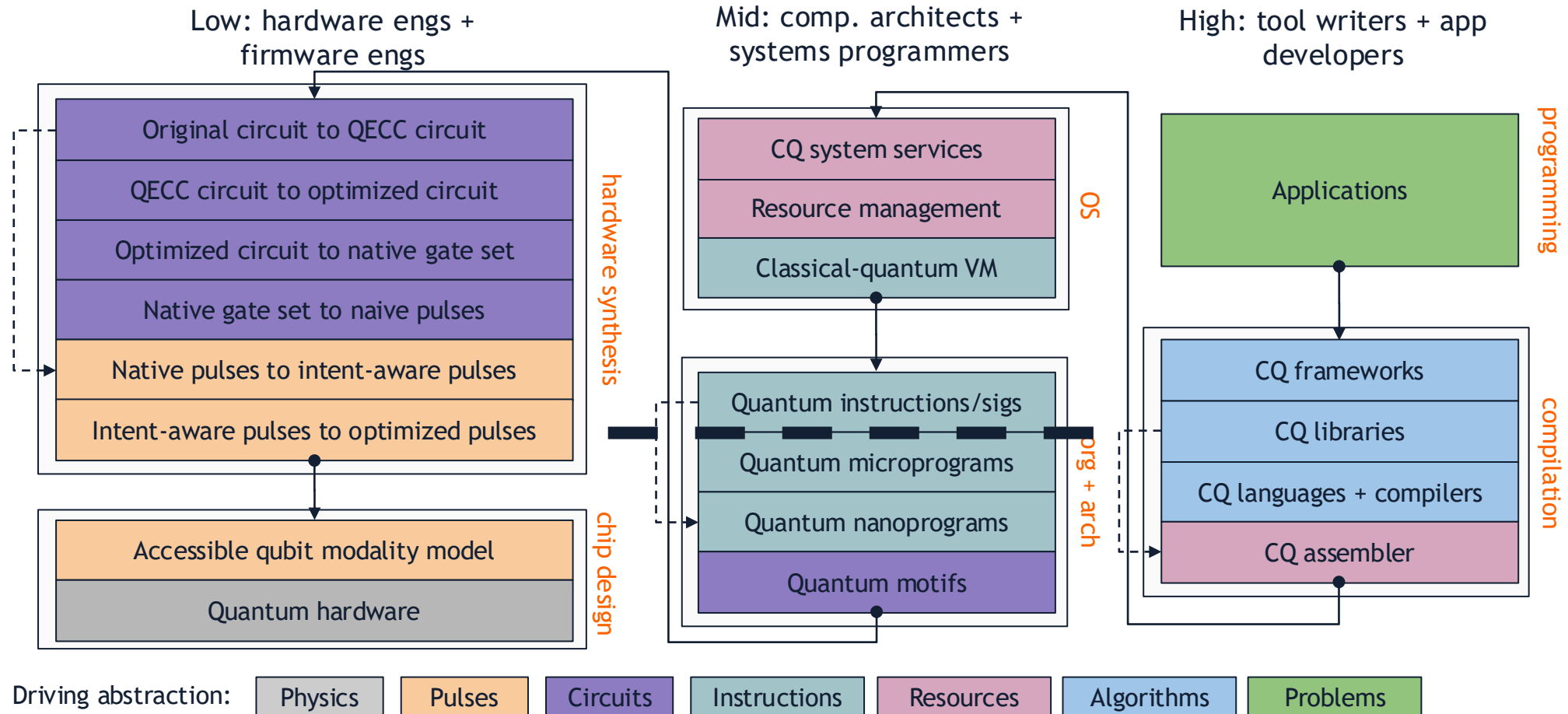
(-2, 9587723626792827)

RZ

(-1, 570796326794897)

c16

Conclusion: we need prescriptive, abstraction-driven design toward HPC-QPU programmability





Plan to throw one (implementation)
away; you will, anyhow.

— Fred Brooks —

= FTQC

AZ QUOTES

